Adrian Buczkowski

West Lafayette, IN | 765-430-2054 | buczkowskiadrian3@gmail.com

Education

Purdue University – B.S. in Computer Engineering May 2026

- **Coursework:** Computer Architecture, ASIC Design, OS, Networks, Compilers, OOP, Microcontrollers, Digital Logic Design, Circuits
- **Skills:** SystemVerilog, RTL, Python, C, C++, Makefile, RISC-V, Linux, Bash
- **Tools:** Design Compiler, QuestaSim, VCS, Virtuoso, KiCAD, MATLAB, FPGA, GitHub

Experience

STARS (Summer Training, Awareness, and Readiness for Semiconductors) Summer 2023, 2024, 2025

Chip Design Project Leader & Intern

- Led teams of students (13 total) through the chip design process. Projects involved developing designs at the RTL, software, and system levels. Selection of peripheral devices was also relevant.
- Ensured student projects were feasible and met performance and area constraints. Several projects involved the creation of CPUs connected to graphical and user input interfaces.
- Developed bus architecture that allowed 8+ student projects to be interconnected on an SoC.
- Wrote System Verilog code to implement and test a variety of designs (CPUs, screen interfaces).
- Successfully generated GDSII files for rigorously verified designs to be taped out. This involved debugging issues related to critical path constraints and manufacturability problems.

Purdue ASIC Design Lab January 2024 – May 2026

Undergraduate Teaching Assistant

- Instructed 360+ students in SystemVerilog-based ASIC design and verification, reinforcing industry standards.
- Debugged 200+ RTL implementations, testbenches, and simulation results to ensure functional correctness.
- Led multiple weekly lab sessions, providing technical mentorship, and clarifying complex concepts in ASIC design.

Research

AI Hardware Senior Design Project: Non-Blocking Memory Subsystem: DDR4 DRAM Controller (SoCET) August 2025 – May 2026

Undergraduate Researcher

- Led the RTL design for a non-blocking memory controller. The design reorders requests to maximize memory bandwidth and reduce memory stalls in an AI accelerator chip.
- Considered power, performance, and area constraints to balance controller performance with the constraints of the team.
- Communicated across several teams to ensure the controller met correctness & performance goals while interacting with the system.
- This project enables the SoCET group to simulate memory access times accurately and yields the possibility of taping out a project with off-chip memory. It achieved a 4x speedup relative to the previous DRAM controller design.

SoCET: Development of FPGA Standard Cells August 2023 – May 2024

Undergraduate Researcher

- Designed transistor optimized standard cells in Cadence Virtuoso to improve area metrics on rad-hard FPGA implementations.
- Achieved a ~7% decrease in individual cell area by improving circuit topologies.

Projects

Multi-Core RISC-V Processor (32 bit) August 2024 – December 2024

- Designed, implemented, and tested a dual-core CPU including a 5-stage pipeline, coherent L1 caches, branch prediction, and a bus controller from scratch in SystemVerilog; successfully synthesized design on FPGA.
- Devised SystemVerilog testbenches and parallel assembly programs to verify and benchmark the processor.
- Made class leaderboards in clock frequency and program run time.

GEM5 Waiting Instruction Buffer August 2025 – December 2025

- Reproduced the results of the popular paper "A Large, Fast Instruction Window for Tolerating Cache Misses" in the GEM5 cycle accurate simulator.
- Wrote 500+ lines of C++ in the GEM5 O3 CPU model to capture the behavior of the WIB and achieve efficient simulation times.
- Results were true to the original paper and showed that a WIB structure can increase ILP in the presence of cache misses.

JOS: Unix-Like Operating System in C January 2024 – May 2024

- Built and debugged critical OS features: virtual memory, user environments, process scheduling, and file systems.
- Created primitives for inter-processor communication using a message passing model to enable parallelism between processes.

Software Defined Network Simulation January 2025 – May 2025

- Used Python to simulate a controller/switches in a SDN as local processes on my machine. The controller periodically calculated a routing table and detected switch failures to rapidly adapt to changes in the network.
- All communication between processes was facilitated by socket programming, and certain processes required multithreading.